

Late Breaking Results: Mixed-Cell-Height Detailed Placement under Multi-Cell Spacing Constraints

Benchao Zhu
School of Microelectronics
Fudan University
Shanghai, China
bczhu22@m.fudan.edu.cn

Zheng Zeng
College of Computer and Data Science
Fuzhou University
Fuzhou, China
211027131@fzu.edu.cn

Jianli Chen
School of Microelectronics
Fudan University
Shanghai, China
chenjianli@fudan.edu.cn

ABSTRACT

As technology scales down, multi-cell spacing constraints are imposed by modern circuit designs. Previous works compromise solution quality to address the problem by transforming it into two-cell spacing constraints. In this paper, we propose a detailed placement algorithm considering multi-cell spacing constraints. First, an SAT-based multi-cell spacing violation reduction method is presented to reduce the number of violations with minimum displacement. Then, a window-based violation elimination method is adopted to resolve all the remaining violations. Finally, we refine the placement result with ILP to reduce the cell displacement. Compared with the state-of-the-art work, experimental results show that our algorithm achieves a 19% improvement in displacement and a 39% reduction in runtime.

ACM Reference Format:

Benchao Zhu, Zheng Zeng, and Jianli Chen. 2024. Late Breaking Results: Mixed-Cell-Height Detailed Placement under Multi-Cell Spacing Constraints. In *61st ACM/IEEE Design Automation Conference (DAC '24)*, June 23–27, 2024, San Francisco, CA, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3649329.3663499>

1 INTRODUCTION

With aggressive device scaling in advanced technology nodes and manufacturing limitations, more complicated design rule constraints, such as cell edge spacing constraints, need to be addressed. The cell edge spacing constraints require a minimum spacing between any two cell edge types. As a generalized version of cell edge spacing constraints, multi-cell spacing constraints have been introduced recently. Figure 1 illustrates the mixed-cell-height placement with multi-cell spacing constraints. If both two-cell spacing constraints are not satisfied, there will be a three-cell spacing violation.

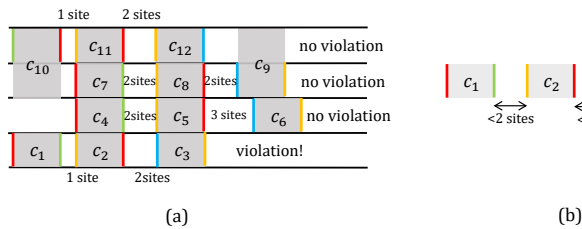


Figure 1: An example of multi-cell spacing constraints. (a) A mixed-cell-height placement with multi-cell-spacing constraints. (b) The multi-cell spacing violation pattern.

This work was partially supported by National Natural Science Foundation of China under Grant 92373207, and the National Key Research and Development Program of China under Grants 2022YFB4400500 and 2021YFA1003602.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. DAC '24, June 23–27, 2024, San Francisco, CA, USA
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0601-1/24/06
<https://doi.org/10.1145/3649329.3663499>

There are several works considering cell edge spacing constraints. [3][5] resolve the two-cell edge spacing constraints in the legalization stage. Due to the complexity of multi-cell spacing constraints, Cheng et al. [2] transform the multi-cell spacing constraints into the critical two-cell spacing constraints to handle the problem in the detailed placement stage. However, these methods compromise solution quality by transforming multi-cell spacing into two-cell spacing.

In this work, we present an effective detailed placement algorithm to address the multi-cell spacing constraints. Compared with the state-of-the-art work, our algorithm achieves 19% smaller displacement and 39% less runtime.

2 PROPOSED METHOD

In this section, we first present an SAT-based multi-cell spacing violation reduction method to reduce violations. Then, we propose a window-based violation elimination method to resolve the remaining violations. Finally, we refine the placement with ILP considering multi-cell spacing constraints.

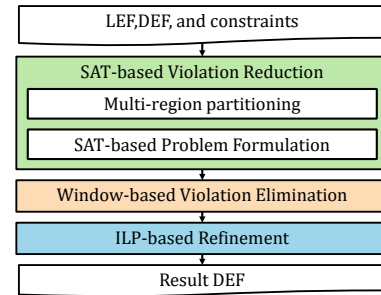


Figure 2: Our algorithm flow.

2.1 Multi-Cell Spacing Violation Reduction

In this part, we aim to minimize the total number of violating cells with minimum displacement. We can formulate the violation reduction problem as a weighted partial maximum satisfiability (MaxSAT) problem. However, the time complexity of the MaxSAT problem will be $O(b2^n)$, where n is the number of distinct variables in the formula and b is the maximum number of occurrences of any variables in the input [4].

2.1.1 Multi-region Partitioning. To reduce the time complexity, we partition the layout into rectangular regions, whose width w_{region} and height h_{region} are defined as follows:

$$w_{region} = \frac{W}{\sqrt{n_{region}}}, h_{region} = \frac{H}{\sqrt{n_{region}}}, \quad (1)$$

where n_{region} is user-specified number of regions, and W and H are the width and height of the layout.

2.1.2 SAT-based Problem Formulation. SAT deals with decision problems, which include hard clauses and soft clauses. The hard clauses are for these constraints that must be satisfied, while the soft clauses can be unsatisfied.

First, we define $\lambda_{c_i}^k$ as the k -th candidate state for cell c_i , and $l(r, p)$ as the p -th site in the r -th row. For each cell, we generate two candidates corresponding to the original orientation and the flipped one in the original sites. If the cell and adjacent cell have the same y coordinate and height

Table 1: Experimental Results.

Benchmark	Statistics					ICCAD'18 [2]			Two-Cell Spacing			Ours		
	Util.	#Cells	#Nets	#Rows	#Violations	Disp. (sites)	Runtime (s)	Violations Remained	Disp. (sites)	Runtime (s)	Violations Remained	Disp. (sites)	Runtime (s)	Violations Remained
sdc	80%	2814	3068	52	309	540	0.68	0	1263	0.52	0	271	0.4	0
aes	80%	8345	8408	86	718	1930	1.29	0	3278	1.38	0	1368	1.16	0
ecg	80%	71632	72176	251	4529	7709	7.65	0	20842	10.77	0	7005	8.68	0
tbp	80%	292701	293481	526	13190	30654	30.72	0	62736	70.18	0	26506	64.72	0
pci	59.7%	29521	29989	200	2438	6012	12.18	0	16439	4.21	0	4694	3.38	0
des	55.0%	112644	112882	300	5123	9907	46.55	0	21579	28.5	0	7463	16.51	0
edit	67.4%	130661	133227	361	5337	9640	60.18	0	16258	27.32	0	8311	19.66	0
Normalized						1.19	1.39	1.00	2.56	1.25	1.00	1.00	1.00	1.00

and are in the same region, we can swap the cell with the adjacent cell and generate candidates corresponding to the original orientation and the flipped one in the sites of the adjacent cell.

Then, the hard clauses are formulated as follows:

- Each cell c_i must choose only one variable $\lambda_{c_i}^k$.
- At most one variable $\lambda_{c_i}^k = 1$ in a site.

The soft clauses can be formulated as follows:

- For violating tuples of $\lambda_{c_i}^m$, $\lambda_{c_j}^n$ and $\lambda_{c_k}^o$, the soft clause can be formulated as $\beta(\bar{\lambda}_{c_i}^m \vee \bar{\lambda}_{c_j}^n \vee \bar{\lambda}_{c_k}^o)$, where β is a user-specified parameter.
- To reduce the displacement between the coordinate of $\lambda_{c_i}^k$ and the legalization coordinate $(\hat{x}_c, \hat{y}_c)$, the soft clause can be formulated as $w_c^k \lambda_{c_i}^k$. The weight w_c^k is defined as follows:

$$w_c^k = \begin{cases} \gamma, & \text{if } \|x_c^k - \hat{x}_c\| + \|y_c^k - \hat{y}_c\| < 1; \\ 1, & \text{otherwise,} \end{cases} \quad (2)$$

where γ is a user-specified parameter.

After formulating our violation reduction problem as a MaxSAT problem in each region, we can use Open-WBO [1] to solve the MaxSAT problem in parallel.

2.2 Window-based Violation Elimination

In Section 2.1, we have resolved most violations with minimum cell displacement. However, violations may exist at the edge of different regions. In this section, we develop a window-based elimination method to resolve all the violations.

We store unresolved violation tuples in a queue format. If the queue is not empty, we pop a violation tuple from the queue. First, we generate a window centered on the violating cell. Based on the window, we devise a data structure called a site map. In the site map, whitespaces are marked as -1 , while the site occupied by a cell is marked as the cell index. Then, we can shift or swap the violating cell to resolve the violation based on the site map. Note that the swappable width range is the violating cell's width to the sum of the violating cell's width and adjacent whitespaces' width.

2.3 ILP-based Refinement

To further refine the placement result, we reduce the displacement without increasing the number of violating cells. First, we collect a set of possible positions O_j for each cell j in m rows. Then, we construct a directed graph to formulate the problem. For each cell and possible position, we add a vertex v_{oj} , where v_{oj} means assigning c_j to position $o \in O_j$. We add source vertex s and target vertex t , and add edges to connect each v_{oj} to the two vertices. For each v_{oj} , if there exists $v_{o'j'}$ in the same row and satisfies $o' \geq o + w_j$ and $j \neq j'$, we add an edge $e = (v_{oj}, v_{o'j'})$. For each edge $(v_{oj}, v_{o'j'})$, we define the cost $c(e)$ as the distance of cell c_j to position o , and $d(e)$ as the number of the critical two-cell spacing violations (most easily satisfied in the violation pattern) from v_{oj} to $v_{o'j'}$.

Finally, we define $x(e)$ as the binary variable to indicate whether edge e is selected. The refinement problem can be formulated as follows:

$$\begin{aligned} \min & \sum c(e)x(e) \\ \text{s.t.} & \sum d(e)x(e) \leq 0 \\ & \sum_{e \in \delta^+(v)} x(e) - \sum_{e \in \delta^-(v)} x(e) = \begin{cases} 0, & v \in V \setminus \{s, t\} \\ 1, & v = s \\ -1, & v = t \end{cases} \\ & \sum_{o \in O_j} \sum_{e \in \delta^+(v_{ij})} x(e) = \frac{h_j}{h_{row}}, \quad \forall c_j \in C \end{aligned} \quad (3)$$

The objective of the ILP is to minimize cell displacement. The first constraint ensures no multi-cell spacing violation. The second constraint restricts that the amount of flow into a vertex equals that out of it unless it is the source or target vertex. The third constraint ensures that all cells are covered in the found paths. The ILP is equivalent to finding m shortest paths by traversing cells in the graph, which can be solved by IBM ILOG CPLEX Optimizer.

3 EXPERIMENTAL RESULTS

We implemented our detailed placement algorithm in the C++ language and tested it on the benchmarks provided by the authors of [2]. Table 1 gives the benchmark statistics, where "Util.", "#Cells", "#Nets", "#Rows" and "#Violations" represent the utilization of the design and the number of standard cells, nets, rows, and multi-cell spacing violations before detailed placement in the design, respectively. Note that the first four designs are single-cell-height designs, while the latter three designs are mixed-cell-height designs.

We compared our detailed placement with the state-of-the-art work ICAD'18 [2] and "Two-Cell Spacing" (our method treating multi-cell spacing constraints as multiple two-cell spacing constraints). With the binary code provided by the author of [2], we were able to run the two algorithms on the same Linux workstation with Intel(R) Xeon(R) Gold 6226 CPU @2.70GHz using 32 threads. The experimental results are shown in Table 1, where "Disp.", "Runtime" and "Violations Remained" give the total displacement, runtime, and remaining violations, respectively. From Table 1, it's clear that all three methods achieve violation-clean placement, and our algorithm produces the best placement results and runtime. Specifically, by treating multi-cell spacing constraints as multiple two-cell spacing constraints, even though "Two-Cell Spacing" gives violation-clean placement results, it produces 156% larger displacement and 25% longer runtime. Compared with ICAD'18 [2], our algorithm achieves 19% better displacement and 39% shorter runtime. Overall, the experimental results show that our algorithm is effective and efficient for addressing the multi-cell spacing constraints.

REFERENCES

- [1] Open-wbo. <https://sat.inesc-id.pt/open-wbo/>.
- [2] Y.-H. Cheng, D.-W. Huang, W.-K. Mak, and T.-C. Wang. A practical detailed placement algorithm under multi-cell spacing constraints. In *2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, page 1–8, 2018.
- [3] H. Li, W. Chow, G. Chen, B. Yu, and E. F. Y. Young. Pin-accessible legalization for mixed-cell-height circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 41(1):143–154, 2022.
- [4] H. Zhang, H. Shen, and F. Manyà. Exact algorithms for max-sat. *Electronic Notes in Theoretical Computer Science*, 86(1):190–203, 2003.
- [5] Z. Zhu, J. Chen, W. Zhu, and Y. Chang. Mixed-cell-height legalization considering technology and region constraints. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 39(12):5128–5141, 2020.